K8s Deep Dive

Pods

A Pod represents a collection of application containers and volumes running in the same execution environment. Pods, not containers, are the smallest deployable artifact in a Kubernetes cluster. This means all of the containers in a Pod always land on the same machine.

Applications running in the same Pod share the same IP address and port space (network namespace), have the same hostname (UTS namespace), and can communicate using native interprocess communication channels over System V IPC or POSIX message queues (IPC namespace).

Applications in different Pods are isolated from each other; they have different IP addresses, different hostnames, and more. Containers in different Pods running on the same node might as well be on different servers.

“What should I put in a Pod?”

Pods

Sometimes people see Pods and think, “Aha! A WordPress container and a MySQL database container should be in the same Pod.” However, this kind of Pod is actually an example of an anti-pattern for Pod construction.

There are two reasons for this: First, WordPress and its database are not truly symbiotic. If the WordPress container and the database container land on different machines, they still can work together quite effectively, since they communicate over a network connection.

Secondly, you don’t necessarily want to scale WordPress and the database as a unit. WordPress itself is mostly stateless, and thus you may want to scale your WordPress frontends in response to frontend load by creating more WordPress Pods. Scaling a MySQL database is much trickier, and you would be much more likely to increase the resources dedicated to a single MySQL Pod. If you group the WordPress and MySQL containers together in a single Pod, you are forced to use the same scaling strategy for both containers, which doesn’t fit well.

In general, the right question to ask yourself when designing Pods is, “Will these containers work correctly if they land on different machines?” If the answer is “no,” a Pod is the correct grouping for the containers. If the answer is “yes,” multiple Pods is probably the correct solution.

Pods

Pods are described in a Pod manifest. The Pod manifest is just a text-file representation of the Kubernetes API object. Kubernetes strongly believes in declarative configuration.

Declarative configuration means that you write down the desired state of the world in a configuration and then submit that configuration to a service that takes actions to ensure the desired state becomes the actual state.

The Kubernetes API server accepts and processes Pod manifests before storing them in persistent storage (etcd). The scheduler also uses the Kubernetes API to find Pods that haven't been scheduled to a node. The scheduler then places the Pods onto nodes depending on the resources and other constraints expressed in the Pod manifests.

Multiple Pods can be placed on the same machine as long as there are sufficient resources. However, scheduling multiple replicas of the same application onto the same machine is worse for reliability, since the machine is a single failure domain.

Consequently, the Kubernetes scheduler tries to ensure that Pods from the same application are distributed onto different machines for reliability in the presence of such failures. Once scheduled to a node, Pods don't move and must be explicitly destroyed and rescheduled.

Creating a Pod (Imperative Approach)

kubectl run nginx --image=nginx:latest --restart=Never

- Kubectl: CLI tool used to interact with Kubernetes
- Run: Creates a resource quickly (imperative)
- Nginx: Name of the Pod
- --image=nginx:latest: Uses the official NGINX container image
- latest = latest version tag
- --restart=Never
- Ensures a Pod is created, not a Deployment
- Default behavior (without this)creates a Deployment

Creating a Pod (Imperative Approach)

kubectl get pods

Lists all Pods in the current namespace

Shows:

- STATUS (Running, Pending, CrashLoopBackOff)
- READY containers
- RESTART count
- AGE

Pod Details: *kubectl describe pod nginx*

Creating a Pod (Declarative Approach)

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
```

kubectl apply -f nginx-pod.yaml

- apiVersion: v1: Defines Kubernetes API version. v1 = core API group
- kind: Pod - Specifies resource type
- Metadata: Identifies the object
- name: nginx-pod - Unique Pod name
- labels: app: nginx
- Key-value pairs: Used for Selection, Grouping, Services
- Spec: Defines the desired state
- Containers: List of containers inside the Pod
- name: nginx - Container name (not Pod name)
- image: nginx:latest - Pulls image from registry. Default registry: Docker Hub
- ports:- containerPort: 80 - Port exposed inside container **Informational (used by tools and Services)**

Accessing Your Pod

kubectl port-forward pod/nginx-pod 8080:80

Local machine port to Pod port: 8080 (local) to 80 (container)

http://localhost:8080

Viewing Logs

kubectl logs nginx-pod

Executing Commands Inside Pod

kubectl exec -it nginx-pod -- /bin/bash

Health Checks

- Kubernetes cannot know if your app is broken
- It assumes everything is fine as long as the process is running
- A container can be: Running but not working
- Kubernetes only sees process = alive, not application = healthy
- Kubernetes supports 3 types (important to distinguish):

1. Liveness Probe: “Should I restart it?”
2. Readiness Probe: “Can it receive traffic?”
3. Startup Probe: “Has it finished booting?”

Liveness Probe (Restart Logic)

YAML:

```
livenessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 5
  periodSeconds: 10
```

httpGet
- Kubernetes sends HTTP request to container
- If response is not 200–399: fail

initialDelaySeconds: 5
Wait 5 seconds after container starts
Prevents killing container too early

periodSeconds: 10
- Check every 10 seconds

If probe fails repeatedly:
Kubernetes kills the container then restarts it

Restart happens inside the Pod, Pod is NOT recreated, Only container restarts

Readiness Probe (Traffic Control)

YAML:

```
readinessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 5
  periodSeconds: 10
```

If readiness fails: Pod is **REMOVED from Service load balancing**
BUT container **keeps running**

Scenario - Imagine:

- App starts slowly (e.g., loading data), Without readiness probe:
- Traffic hits app immediately = errors
- With readiness probe: Kubernetes waits until ready

Resource Management

YAML:

```
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "256Mi"
```

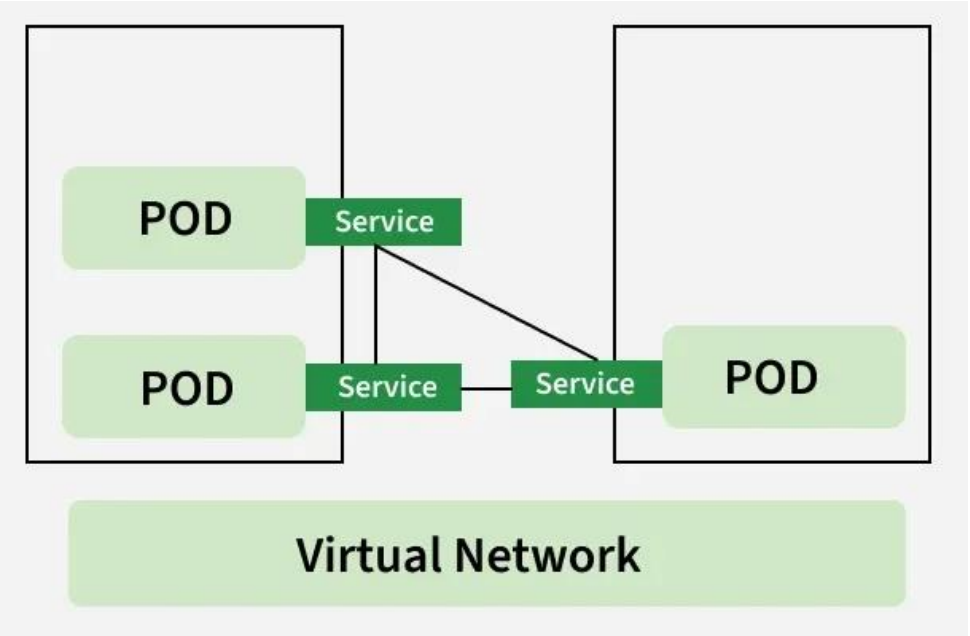
- Kubernetes is a shared system: Many Pods but Limited CPU & memory
- Without it: One Pod can consume ALL CPU, Node crashes and Other Pods fail
- **requests (Scheduling Guarantee)**
  - Minimum required resources
  - Kubernetes uses this to:
    - Decide where to place Pod
  - Example:
    - Node has 1 CPU
    - Pod requests 0.5 CPU → OK
    - Pod requests 2 CPU → NOT scheduled
- **limits (Enforcement):** Maximum allowed usage
- CPU limit: Container is throttled
- Memory limit: Container is killed

100m = 0.1 CPU
Mi = Mebibyte
Gi = Gibibyte

Kubernetes Services

A Kubernetes Service is a stable networking abstraction that exposes a group of Pods as a single network endpoint. Pods in Kubernetes are ephemeral — they can restart, crash, or move across nodes. Pod IP addresses are not permanent, which makes direct communication unreliable, applications in a cluster need a stable way to communicate with each other. Kubernetes Services solve this problem by providing:

- Stable virtual IP (ClusterIP).
- Built-in load balancing.
- Service discovery via DNS.
- Decoupling between clients and Pods.



Types of Services - ClusterIP

A ClusterIP is the default type of Service used to expose an application inside the cluster only.

A ClusterIP is a virtual IP address assigned to a Service that:

- Allows pods to communicate with each other
- Is only reachable from within the cluster
- Is managed by Kubernetes only

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
```

type: ClusterIP: Default Service type. Exposes the service only inside the cluster.

selector: app: nginx: Routes traffic to Pods with the label app=nginx.

port: 80 The port on which the Service is exposed internally.

targetPort: 80 The container port inside the Pod where NGINX is running.

Types of Services -NodePort (External Service)

In NodePort Service, external service has access to a fixed port on each Worker Node. In this case, instead of ingress, the browser request will directly come to the node at the port defined in the service config file and be exposed at the node port. The node port can range between 30000 to 32767.

- type: NodePort: Exposes the service on each Node's IP at a static port (30000).
- port: 80: Service port inside the cluster.
- targetPort: 80: Container port inside the Pod.
- nodePort: 30000: External port accessible via <NodeIP>:30000

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30000
```

Types of Services - LoadBalancer

Exposes the service externally using a cloud provider's load balancer. NodePort and ClusterIP services, to which the external load balancer will route, are automatically created. If you are using a custom Kubernetes Cluster (using minikube, kubeadm, or the like). In this case, there is no LoadBalancer integrated (unlike AWS EKS or Google Cloud, KOPS, and AKS). With this default setup, you can only use NodePort.

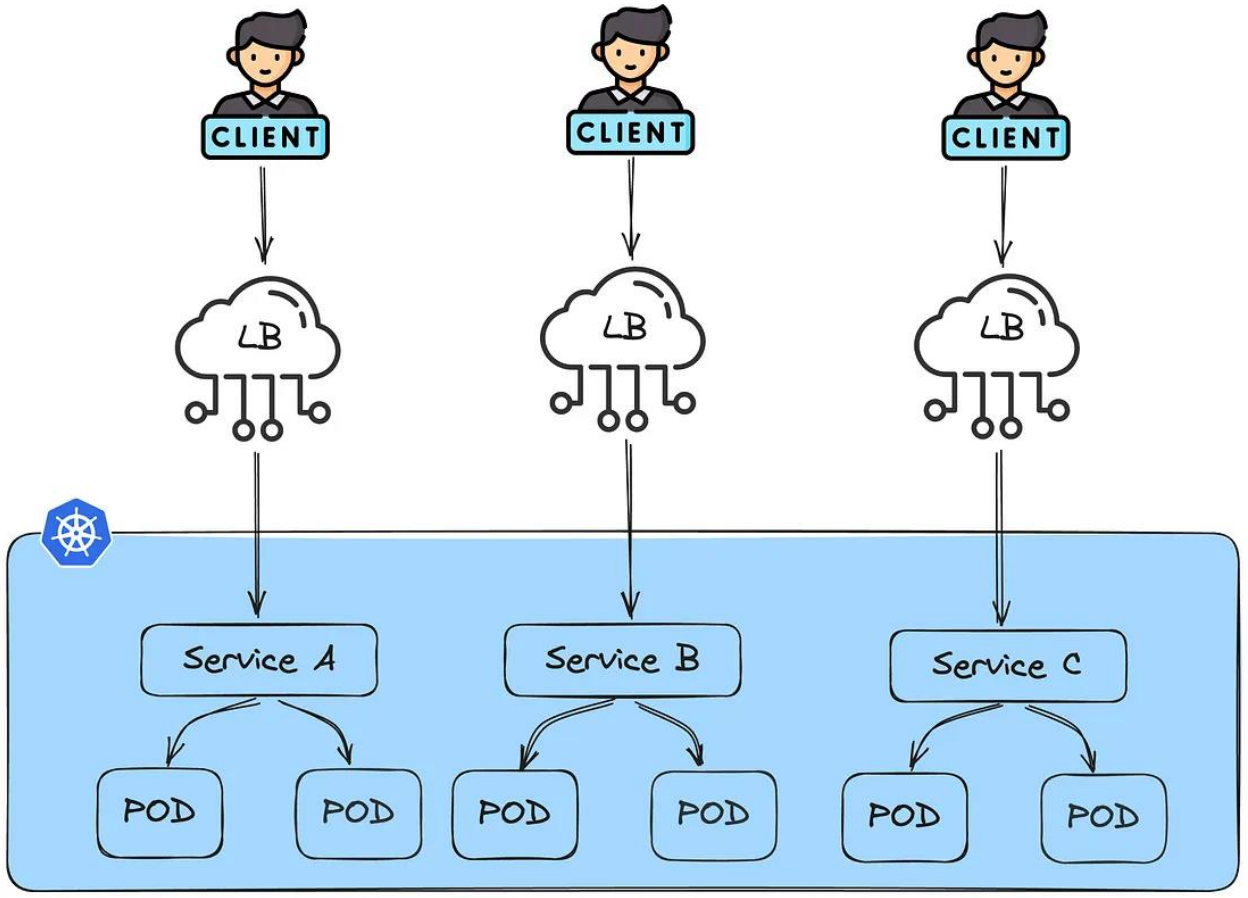
- type: LoadBalancer: Exposes the service externally using a cloud provider's load balancer.
- port: 80: Port exposed by the Service.
- targetPort: 8080: Port on which the container inside the Pod is running.
- selector: app: javawebapp: Routes traffic to Pods with this label.

```
apiVersion: v1
kind: Service
metadata:
  name: javawebappsvc
spec:
  type: LoadBalancer
  selector:
    app: javawebapp
  ports:
    - port: 80
      targetPort: 8080
```

Ingress Controller

imagine if we have a microservices architecture. Here's how the final result would look.

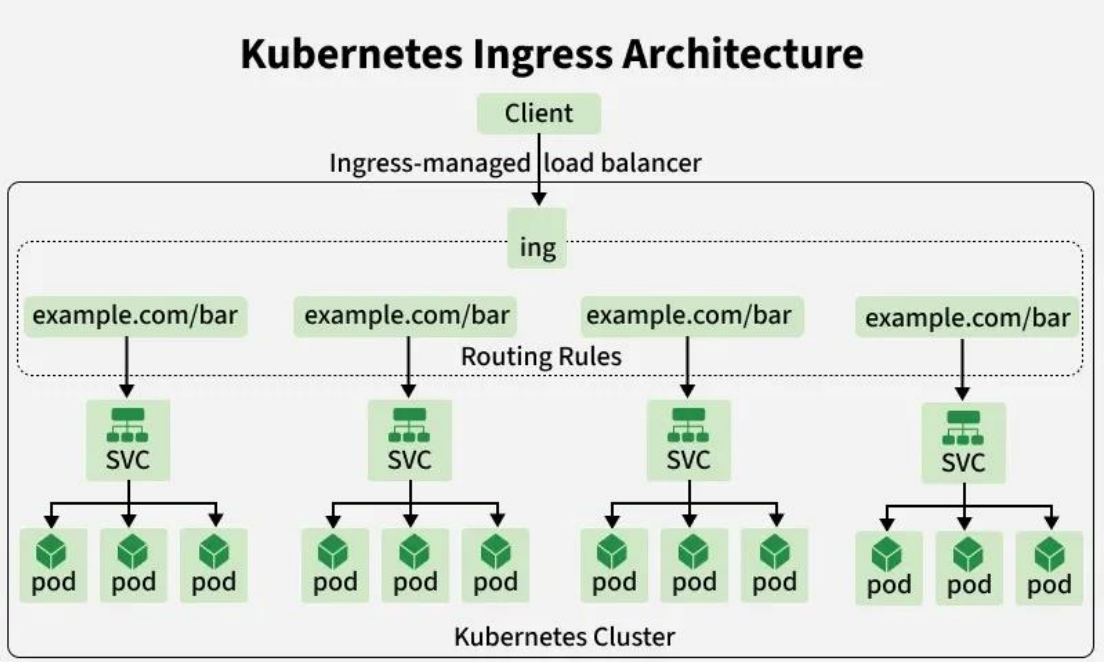
This architecture can become costly because it requires a dedicated load balancer for each service. Therefore, the need for a centralized solution that handles all traffic at a single point arises. This is where Ingress comes into play.



Ingress Controller

Ingress is a Kubernetes API object that exposes HTTP and HTTPS routes from outside the cluster to services running inside. It provides a single entry point for managing external access, simplifying application management and routing. Kubernetes Ingress supports:

- Routes traffic to services based on hostnames or URL paths.
- Supports TLS/SSL termination for secure connections.
- Enables load balancing for multiple backend pods.
- Requires an Ingress Controller (e.g., NGINX, Traefik) to implement the routing rules.
- Helps consolidate multiple service endpoints under a single IP or domain.



Ingress Controller

Key Components of Kubernetes Ingress

- Ingress Resource: Defines the routing rules for directing external traffic to internal services. This component holds all the configuration details like host-based and path-based routing.
- Ingress Controller: Implements the rules defined in the Ingress Resource. It watches for changes and updates, ensuring traffic flows according to the defined configuration. Popular Ingress Controllers include NGINX, HAProxy, and Traefik.
- Backend Services: The services receiving the traffic directed by Ingress. These could be any internal applications or APIs within your Kubernetes cluster.

Ingress Controller

There are two main types of Ingress controllers, each with completely different implementation models.

- Ingress based on a web server
- Ingress based on a cloud load balancer

Ingress based on a Web server :

This type of Ingress controller is based on an actual web server such as NGINX or HAproxy. Behind the scenes, when this type of controller is deployed, it starts an NGINX web server in the cluster. The controller's job is to convert the Ingress configuration template provided in the Ingress manifest to an actual configuration for the web server and update the generic configuration file.

Ingress Controller

For example, consider this Ingress manifest:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: example-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - host: example.com
    http:
      paths:
      - path: /app
        pathType: Prefix
        backend:
          service:
            name: app-service
            port:
              number: 80
```

The equivalent configuration in Nginx is a s below:

```
http {
  server {
    server_name example.com;

    location /app {
      proxy_pass http://app-service:80;
    }
  }
}
```

Ingress Controller

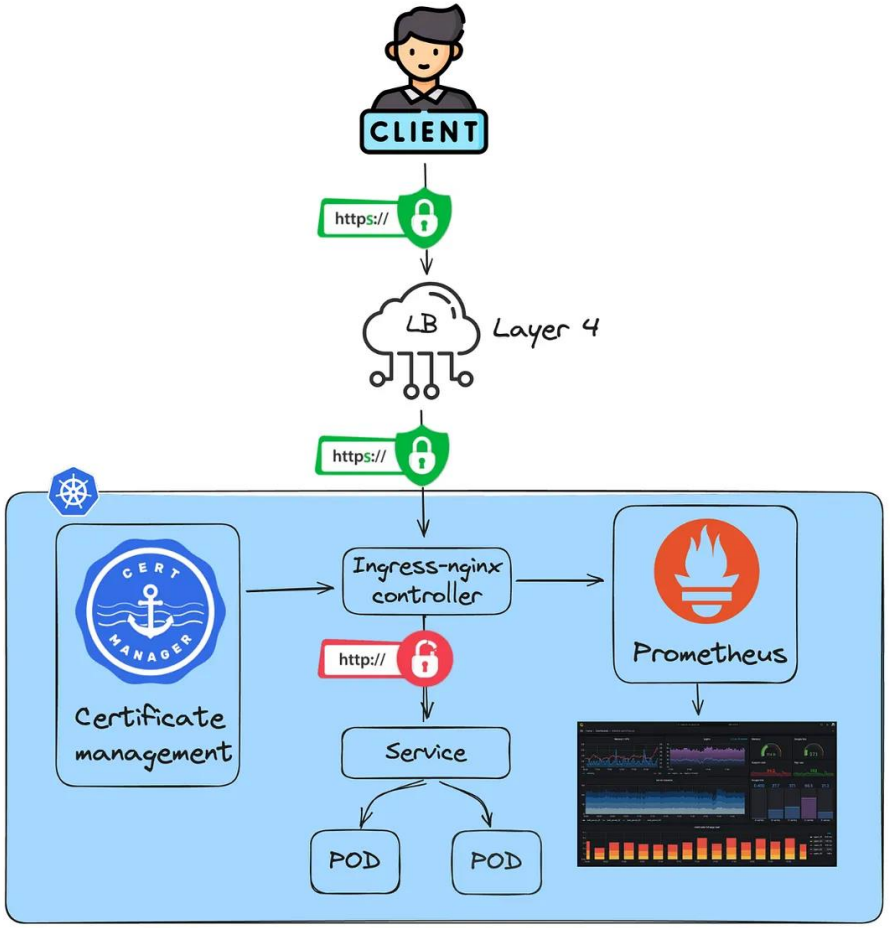
As shown in the figure above, for this type of Ingress controller we need a layer 4 (transport layer) load-balancer and the Ingresss web-server , in this example Nginx, will handle the application network traffic.

This approach has various advantages and disadvantages. In summary here is how it works under the hood and here are some of them:

Pros :

- Kubernetes Native solution and centrelized configuration.
- TLS management, We can deploy Certificate management tools to manage certificate lifeCycle and advanced security feature in kubernetes cluster itself.
- Monitoring, we can expose and handle in cluster level diffrent request signals (Latency, traffic, errors…)

Cons:



Ingress Controller

Pros :

- Kubernetes Native solution and centrelized configuration.
- TLS management, We can deploy Certificate management tools to manage certificate lifeCycle and advanced security feature in kubernetes cluster itself.
- Monitoring, we can expose and handle in cluster level diffrent request signals (Latency, traffic, errors…)

Cons:

- Resource Consumption: while it is been deployed to kubernetes we need to consider a high resource consumption for load balancing , routing rules execution and other configuration.
- Maintenance Overhead: It is not a manged service so we need to manage upgrades and support by ourselves.

Ingress Controller

Ingress based on a cloud load balancer

On the other hand, Cloud Load Balancer Ingress controllers depend on cloud-managed services. Essentially, all load balancing processes are managed by provisioned load balancers in the cloud, and the traffic is directed by the Cloud Controller Manager to the targets. In this implementation mode, we operate outside of the Kubernetes context, as all configurations are made in a different type of load balancer external to the cluster.

Let’s consider a concrete example from one of the most widely used cloud providers, AWS. AWS has implemented an Ingress controller based on AWS Application Load Balancer (ALB), known as “aws-alb-ingress-controller”:

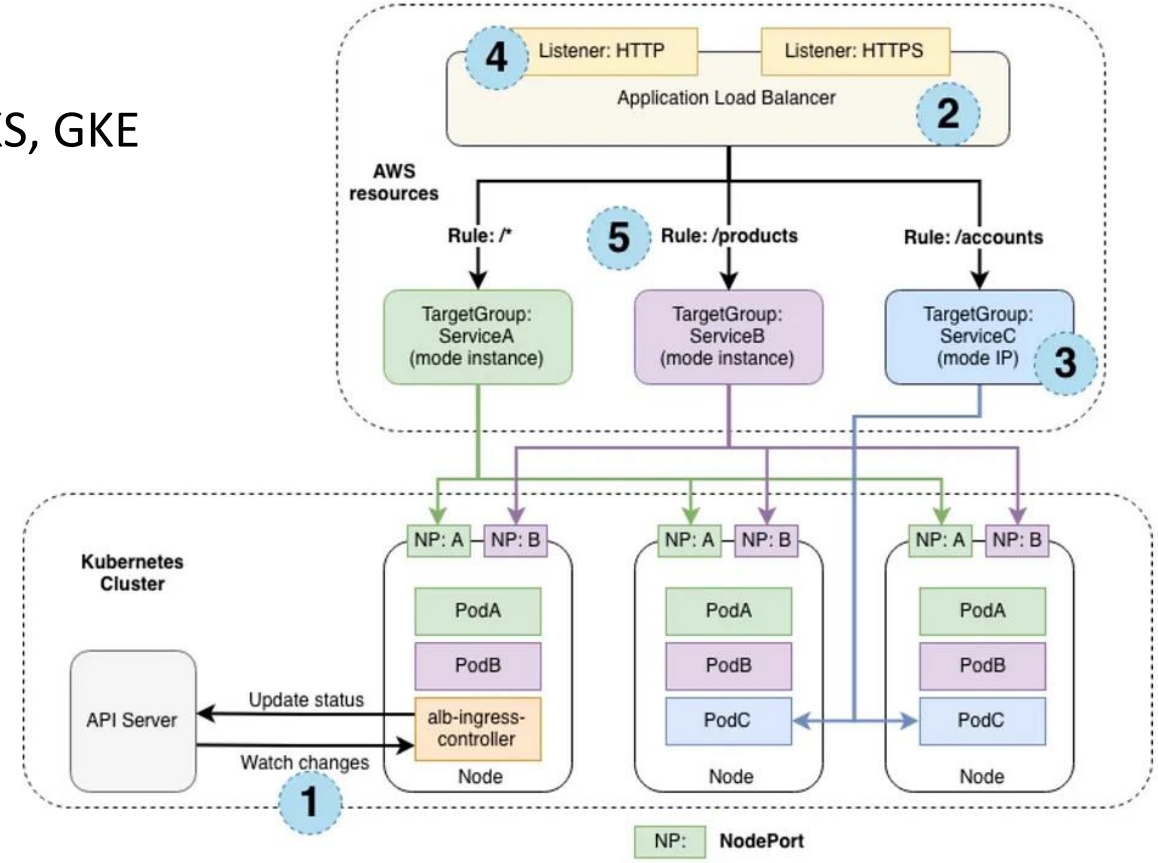
Ingress Controller

Pros

- Native integration in managed clusters (EKS, AKS, GKE …).
- Managed Service: No Maintenance and less responsibility.
- High availability: while it uses AWS ALB so it is autoscaling.

Cons

- Cost: managed Services are pricy.
- Limited accessibility and needs cloud provider knowledge.



Replicaset

ReplicaSet is used to maintain a certain number of pods always running all the time. ReplicaSet always monitors the current state and desired state of pods. ReplicalSet ensure a stable set of healthy pods running within a cluster at any given time. In the ReplicaSet configuration file we mainly mention two important features, number of replicas to maintain desired replica count and a pod template for creating new pods. On any pod failure or deletion of pod , replicaset automatically creates another healthy pod in the cluster to maintain the desired number of replicas. This feature of ReplicaSet ensures high availability.

The ReplicaSet configuration file consists of the following fields :

- ApiVersion : It describes the Kubernetes API version that supports ReplicaSet .
- Kind : It describe what type of resource . For example RepliceSet
- Replicas : It describes the number of desired replicas .
- Selector : It describes the group of pod .
- Template : It describes which image and container port is used to create a pod .

Replicaset

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: gfg-nginx-replica
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx #manage pods that have this label
  template:
    metadata:
      labels:
        app: nginx #pods label
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
```

Deployments

Kubernetes Deployments are widely used to manage application lifecycles in a cluster. Typical use cases include:

- Rolling out new applications → Create a Deployment that launches a ReplicaSet, which in turn provisions Pods in the background. You can monitor the rollout status to verify success.
- Updating applications seamlessly → Modify the PodTemplateSpec in a Deployment to trigger a new ReplicaSet. The Deployment gradually scales the new ReplicaSet up while scaling the old one down, replacing Pods at a controlled pace. Each update increments the Deployment's revision.
- Rolling back safely → Revert to a previous Deployment revision if the current rollout fails or becomes unstable. Each rollback generates a new revision.
- Scaling with demand → Increase or decrease replicas in a Deployment to handle varying traffic loads.
- Pausing and resuming rollouts → Temporarily pause a Deployment to apply multiple fixes to the Pod template, then resume it to roll out all changes at once.
- Monitoring rollout status → Use the Deployment's status field to detect if a rollout is progressing or stuck.
- Cleaning up resources → Remove outdated ReplicaSets that are no longer needed to keep the cluster tidy and efficient.

Deployments

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx
        resources:
          limits:
            memory: "128Mi"
            cpu: "500m"
        ports:
        - containerPort: 80
```